

M12 / PROTECTED ANSWERS

M12 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	8
Open only after a genuine saved attempt	8
DE12-01 Why Spark exists: driver, executors, partitions and tasks	8
What strong evidence contains	8
Reasoning to repair	8
Most important failure to catch	8
Professional boundary	8
Fresh retry	8
Do not call Mastered yet if...	8
DE12-02 PySpark setup, SparkSession and laptop-safe local mode	9
What strong evidence contains	9
Reasoning to repair	9
Most important failure to catch	9
Professional boundary	9
Fresh retry	9
Do not call Mastered yet if...	9
DE12-03 DataFrames and explicit schemas	9
What strong evidence contains	9
Reasoning to repair	10
Most important failure to catch	10
Professional boundary	10

Fresh retry	10
Do not call Mastered yet if...	10
DE12-04 Spark SQL and expression equivalence	10
What strong evidence contains	10
Reasoning to repair	10
Most important failure to catch	11
Professional boundary	11
Fresh retry	11
Do not call Mastered yet if...	11
DE12-05 Lazy transformations and actions	11
What strong evidence contains	11
Reasoning to repair	11
Most important failure to catch	11
Professional boundary	11
Fresh retry	12
Do not call Mastered yet if...	12
DE12-06 Jobs, stages, tasks and physical plans	12
What strong evidence contains	12
Reasoning to repair	12
Most important failure to catch	12
Professional boundary	12
Fresh retry	12
Do not call Mastered yet if...	12

DE12-07 repartition vs coalesce	13
What strong evidence contains	13
Reasoning to repair	13
Most important failure to catch	13
Professional boundary	13
Fresh retry	13
Do not call Mastered yet if...	13
DE12-08 Aggregations and shuffle cost	13
What strong evidence contains	13
Reasoning to repair	14
Most important failure to catch	14
Professional boundary	14
Fresh retry	14
Do not call Mastered yet if...	14
DE12-09 Joins and broadcast strategy	14
What strong evidence contains	14
Reasoning to repair	14
Most important failure to catch	15
Professional boundary	15
Fresh retry	15
Do not call Mastered yet if...	15
DE12-10 Join correctness, fan-out and reconciliation	15
What strong evidence contains	15

Reasoning to repair	15
Most important failure to catch	15
Professional boundary	15
Fresh retry	16
Do not call Mastered yet if...	16
DE12-11 Data skew	16
What strong evidence contains	16
Reasoning to repair	16
Most important failure to catch	16
Professional boundary	16
Fresh retry	16
Do not call Mastered yet if...	16
DE12-12 Adaptive Query Execution (AQE)	17
What strong evidence contains	17
Reasoning to repair	17
Most important failure to catch	17
Professional boundary	17
Fresh retry	17
Do not call Mastered yet if...	17
DE12-13 Caching and persistence	17
What strong evidence contains	17
Reasoning to repair	18
Most important failure to catch	18

Professional boundary	18
Fresh retry	18
Do not call Mastered yet if...	18
DE12-14 Parquet I/O and partition pruning	18
What strong evidence contains	18
Reasoning to repair	18
Most important failure to catch	18
Professional boundary	19
Fresh retry	19
Do not call Mastered yet if...	19
DE12-15 Built-in functions vs Python UDFs	19
What strong evidence contains	19
Reasoning to repair	19
Most important failure to catch	19
Professional boundary	19
Fresh retry	19
Do not call Mastered yet if...	20
DE12-16 Driver safety: collect, toPandas and bounded inspection	20
What strong evidence contains	20
Reasoning to repair	20
Most important failure to catch	20
Professional boundary	20
Fresh retry	20

Do not call Mastered yet if...	20
DE12-17 Testing and debugging Spark transformations	20
What strong evidence contains	20
Reasoning to repair	21
Most important failure to catch	21
Professional boundary	21
Fresh retry	21
Do not call Mastered yet if...	21
DE12-18 Integrated PySpark pipeline and distributed-processing checkpoint	21
What strong evidence contains	21
Reasoning to repair	21
Most important failure to catch	22
Professional boundary	22
Fresh retry	22
Do not call Mastered yet if...	22

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a genuine saved attempt

DE12-01 | Why Spark exists: driver, executors, partitions and tasks

What strong evidence contains

The simulator says the driver plans/co-ordinates, executors run tasks over partitions, and local[2] is two local task threads. The Spark lab should report 18 source rows. Partition counts can differ by reader and environment; the lesson is to inspect them rather than memorise a number.

Reasoning to repair

Spark can parallelize because the logical dataset is split into partitions and tasks can operate on independent partitions. The driver does not personally process every row just because Python started the application. Keeping this distinction prevents later errors such as bringing all data back to the driver with `collect()`.

Most important failure to catch

Do not say 'a partition is a server' or 'one row equals one task.' One executor can process many partitions over time, and one partition can contain many rows. Also do not infer a production cluster from local[2]. Local mode is a useful execution engine on one machine, not distributed hardware.

Professional boundary

Use the execution model to explain evidence, not as vocabulary theatre. For every Spark incident be able to state where the code runs, which operation created the partition boundary, what one output row means and which row/key/total check proves business correctness.

Fresh retry

A stage has 80 partitions and the cluster has 8 executor cores available. Explain why 80 tasks do not run simultaneously and why reducing to 8 partitions automatically is not always the right repair.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-02 | PySpark setup, SparkSession and laptop-safe local mode

What strong evidence contains

Your learner environment should report PySpark 4.2.0, a supported Python version and Java 17 or later. A SparkSession created through `common.spark_session()` should use `local[2]`, 1g driver memory, `spark.sql.shuffle.partitions=4` and `AQE true`. Record the actual values rather than assuming configuration took effect.

Reasoning to repair

A pinned small environment separates course logic from machine noise. When every lab uses the same session builder, a plan difference is less likely to be caused by one forgotten setting. Explicitly stopping sessions also reduces the chance that multiple JVMs compete for your limited RAM.

Most important failure to catch

Common setup mistakes are mixing system Python and venv Python, installing multiple Spark versions, assuming Java 8/11 guidance from an old video is current, or opening Airflow/Docker/PostgreSQL/Spark together on an 8 GB laptop. Another mistake is treating `pip install pyspark` success as proof that a Spark job actually starts.

Professional boundary

A trustworthy runtime record says what was really executed on which environment. Never turn static code inspection into a live-Spark PASS. Stable setup and honest runtime evidence make later performance observations comparable.

Fresh retry

Suppose a tutorial uses `local[*]` and an 8g driver. Explain why blindly copying it onto your laptop could make the lesson less reliable, and how you would choose a safer teaching configuration without pretending it is production tuning.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-03 | DataFrames and explicit schemas

What strong evidence contains

The schema should show typed date/integer/double fields instead of all strings. The resulting DataFrame keeps one row per source order and adds `gross_sales` without collecting the whole dataset into Python. The source fixture remains 18 rows.

Reasoning to repair

Built-in column expressions remain part of Spark's structured plan. Explicit types let Spark validate operations and choose encodings without first treating everything as Python objects. More importantly, the data contract becomes reviewable: another engineer can see what the pipeline expects before a bad batch reaches a downstream join.

Most important failure to catch

Do not assume explicit schema means bad values are automatically quarantined with useful reasons; input error behavior still needs a contract. Do not mark every field nullable just to avoid errors. And do not confuse schema with grain: a perfect schema cannot tell you whether one row is one order, item or event unless you state it.

Professional boundary

Prefer explicit schemas at stable ingestion boundaries and built-in expressions for transformations. Save `printSchema()` evidence, row grain and a small bounded sample; the schema alone is not proof that all business values are valid.

Fresh retry

The partner adds a nullable `carrier_code` column. Explain how you would update the schema deliberately and how you would distinguish compatible additive change from an unexpected rename of `customer_id`.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-04 | Spark SQL and expression equivalence

What strong evidence contains

Both routes produce four regional rows with identical totals; their sum is 25,230. The result grain is one row per region. If one route includes Pending/Cancelled rows or uses a different calculation, matching syntax style is irrelevant because business semantics differ.

Reasoning to repair

DataFrame and SQL expressions enter Spark's structured optimizer. Translating between them makes assumptions visible: WHERE/filter defines the population, groupBy/GROUP BY changes the grain, and aggregate expressions define measures. Sorting is added only to make evidence comparisons deterministic, not because DataFrames are inherently ordered.

Most important failure to catch

A common false comparison is `df.show()` versus a SQL query with `ORDER BY` and assuming row-order difference means value mismatch. DataFrames are not guaranteed ordered unless you request ordering. Another mistake is registering a view before adding `gross_sales`, then writing SQL that assumes the column already exists.

Professional boundary

Be bilingual enough to review both APIs, but validate business equivalence with row counts, keys and totals. Matching screenshots are weak evidence; deterministic comparison of values and schema is stronger.

Fresh retry

Rewrite a DataFrame filter for orders on/after 2026-09-03 into SQL. Then explain which part is business logic and which part only controls output ordering.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-05 | Lazy transformations and actions

What strong evidence contains

Before `show()`, you have a DataFrame plan object, not four already-calculated regional totals sitting in Python memory. The action produces the four totals. The exact number of jobs/stages can depend on Spark/version/operation; inspect runtime evidence rather than promising one fixed count from the source code alone.

Reasoning to repair

Laziness gives Spark room to optimize a whole structured plan: it can push filters, prune columns or change join strategy rather than executing each line as an isolated Python instruction. The cost is conceptual: source code order is not a timeline of physical execution.

Most important failure to catch

Do not say 'nothing happens' before an action in an absolute sense: analysis/planning and metadata work can occur, and some operations may interact with catalog/source metadata. The important boundary is that full data computation is deferred. Also do not call `count()` casually after every debugging line; each action can trigger real work.

Professional boundary

Plan first, trigger work deliberately, and record which actions exist. Do not equate Python line count with Spark work. Use `explain`, Spark UI/runtime metrics and independent result controls together.

Fresh retry

A teammate adds `df.count()` only to print 'loaded successfully' before every stage. Explain when that check could be useful and when it is redundant expensive work. Propose a more targeted invariant.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-06 | Jobs, stages, tasks and physical plans

What strong evidence contains

The plan should show a distributed aggregation path with an Exchange/shuffle boundary for grouping by region. Runtime stage/task counts depend on partitions and adaptive execution; do not hard-code a universal number into your explanation.

Reasoning to repair

Rows with the same group key can begin in different input partitions. To compute a final per-region sum, Spark must bring compatible key groups together. That cross-partition dependency creates a stage boundary. The physical plan gives a hypothesis; runtime statistics tell you how the actual execution behaved.

Most important failure to catch

Do not label every operation a shuffle. Simple select/filter/withColumn can often remain narrow. Conversely, do not assume a groupBy is cheap because the result has only four rows; many source rows may have moved before those four results appeared. Small output is not small work.

Professional boundary

Use plan evidence as a map, not fortune-telling. Pair `explain('formatted')` with actual runtime metrics and result reconciliation. A faster wrong plan is still wrong.

Fresh retry

A query adds `orderBy(gross_sales desc)` after `groupBy`. Explain why ordering can add another redistribution/sort requirement and why the exact stage count should be observed rather than guessed.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-07 | repartition vs coalesce

What strong evidence contains

Both routes can report the requested final counts, but the repartition plan should make redistribution visible while coalesce reduction can use a narrower path. Exact plan nodes can evolve; explain the semantic distinction, not a brittle string match.

Reasoning to repair

Parallel work is partition-driven, so too few partitions can underuse available cores and too many tiny partitions can create scheduling/file overhead. Repartition is justified when distribution/key layout needs to change; coalesce is useful when you mainly need fewer partitions after a shrinking operation.

Most important failure to catch

`repartition(1)` before every write is a classic anti-pattern: it serializes output through one partition and can overload one task. The opposite mistake is forcing thousands of partitions because 'more parallelism is faster.' On an 18-row lab both choices can look harmless; production evidence is what matters.

Professional boundary

Change partitioning for a stated reason, inspect the plan/runtime effect, and revalidate rows/keys/totals. Physical tuning must not silently change business semantics.

Fresh retry

After a strong filter leaves 2% of rows, the DataFrame still has hundreds of partitions. Compare coalesce versus repartition for the final write and state when the cheaper narrow reduction is attractive.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-08 | Aggregations and shuffle cost

What strong evidence contains

The grouped totals match the independent Python validator exactly. The Spark plan shows redistribution appropriate to a keyed aggregation. The four-row output is not evidence that only four records were processed.

Reasoning to repair

Filtering to Completed before grouping reduces the population from 18 to 16. Projecting only needed columns can reduce data width. Those correctness-preserving reductions occur before an expensive boundary. But do not manually micro-optimize without measuring: Catalyst can already push many filters/projections.

Most important failure to catch

Two dangerous shortcuts are increasing shuffle partitions blindly and collecting raw rows to aggregate in Python. The first may add overhead; the second defeats distributed execution and risks driver memory. Another error is changing the aggregation grain while 'optimizing', such as grouping by region+category when the requested output is region only.

Professional boundary

Do not fear shuffle; make it intentional. Reduce irrelevant data early, preserve the requested grain, inspect Exchange/runtime evidence and reconcile totals after every distributed aggregation.

Fresh retry

The business asks for top 100 customers by sales. Explain why ``groupBy customer -> sum -> orderBy desc -> limit 100`` can involve more than one expensive distributed boundary and what you would inspect.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-09 | Joins and broadcast strategy

What strong evidence contains

With a unique dimension, the left/fact grain remains one row per Completed order: 16 rows. The plan should show a broadcast join strategy in this tiny lab. Real automatic choices depend on statistics, size, AQE and configuration, so production strategy must be verified from the actual plan.

Reasoning to repair

Broadcast can avoid shuffling the large relation because each executor has local access to the small lookup. The trade-off is replicated memory/network cost for the broadcast side. The engineering decision therefore depends on real size and reuse, not table naming conventions such as 'dimension'.

Most important failure to catch

Never broadcast because a table is called `dim_customer`. Check size. Never use a broadcast hint to hide a many-to-many join. And do not assume the same join strategy is optimal forever: a table can grow, stats can become stale and AQE can adapt decisions at runtime.

Professional boundary

Validate join grain and key uniqueness before tuning. Treat broadcast as a strategy backed by size/plan evidence, not a rule attached to dimension tables.

Fresh retry

The lookup becomes 60 MB compressed on disk but 300 MB in memory. Explain why file size alone is a poor basis for a broadcast decision and which Spark statistics/runtime evidence you would inspect.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-10 | Join correctness, fan-out and reconciliation

What strong evidence contains

The bad contact join has more than 16 rows because C02/C06 duplicate. Its gross-sales sum exceeds 25,230. That is expected failure evidence. The correct repair depends on the business requirement: aggregate contacts to customer grain, choose a valid current contact by explicit rule, or keep order-contact grain and never sum order-level measures as if still one row/order.

Reasoning to repair

Spark's join operator obeys relational matching. If a left key has L matches and right key has R matches, that key can generate $L \times R$ joined combinations. Reconciliation catches the result even when every individual row looks plausible. This is why `count`, `countDistinct(order_id)` and `sum(gross_sales)` belong together after sensitive joins.

Most important failure to catch

Do not use `dropDuplicates()` after the join as a universal fix: which contact row survives can be arbitrary and you may discard legitimate facts. Do not use `sum(distinct gross_sales)` either; two genuine orders can have the same amount. Repair at the grain/key rule, not at the appearance of duplicated values.

Professional boundary

Carry relational discipline into Spark. Every important join should have declared left/right grain, expected cardinality, unmatched policy, key checks and at least one row/key/measure reconciliation.

Fresh retry

A product dimension unexpectedly has two active rows for P09. Describe a pre-join uniqueness test and a post-join count/amount reconciliation that would stop the pipeline before serving wrong totals.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-11 | Data skew

What strong evidence contains

The pure-Python control reports a 0.9 HOT share. The Spark output shows HOT=900. If task-level metrics are too small to look dramatic locally, record that honestly. You have proved the skew shape, not a production slowdown.

Reasoning to repair

Hash partitioning sends equal keys together by design, which is exactly what grouping/join correctness needs. That means a dominant key can dominate one partition. Some remedies change execution strategy (AQE skew splitting), some change data shape (salting), and some change the business model (process giant tenants separately).

Most important failure to catch

Random repartition without a key can redistribute rows for one stage but the next groupBy/join on HOT must bring the key back together. Salting without a second-stage recombination changes the result. And treating the longest task as skew without checking data distribution can confuse slow I/O or GC with hot keys.

Professional boundary

Diagnose skew from distributions and runtime metrics. Choose a remedy that preserves the aggregation/join semantics, and prove both performance change and unchanged business totals.

Fresh retry

The HOT key is required to be processed as one logical group for a non-associative operation. Explain why naive salting may be invalid and what architecture alternatives you would consider.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-12 | Adaptive Query Execution (AQE)

What strong evidence contains

The course profile reports AQE true. You should be able to explain at least three adaptive capabilities and one limit: AQE reacts within supported plan structures; it does not repair wrong keys, missing filters, incorrect grain or every skew problem.

Reasoning to repair

Estimates before execution are imperfect. Runtime map-output statistics reveal actual partition sizes and relation sizes, allowing Spark to make safer choices. This reduces the need to guess a perfect static shuffle-partition count for every stage, while still requiring a sensible initial plan and resource envelope.

Most important failure to catch

Do not say AQE 'automatically makes Spark fast.' It can add planning benefits only where supported and may leave root causes untouched. Do not compare elapsed time from one tiny local run as proof. Warm caches, JVM startup and background load can dominate classroom timings.

Professional boundary

Keep AQE enabled unless you have evidence-based reasons otherwise, but treat it as one layer of execution optimization. Validate data distribution, join correctness and output invariants independently.

Fresh retry

Suppose AQE coalesces many tiny post-shuffle partitions but one hot key still dominates a task. Explain why both facts can be true and what next evidence you need.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-13 | Caching and persistence

What strong evidence contains

The business results still reconcile to 25,230. Cache should appear only after an action materializes it. After unpersist, the explicit cache is released. On this tiny dataset elapsed time is not meaningful evidence; the lab demonstrates lifecycle and reuse reasoning.

Reasoning to repair

Without persistence, separate actions may replay the upstream lineage. Caching stores computed partitions so reuse can avoid some recomputation. But blocks occupy memory/disk according to storage level and may be evicted. The benefit depends on cost to recompute, reuse count, size and available memory.

Most important failure to catch

Caching every intermediate DataFrame is an anti-pattern. It can crowd useful data, increase GC and make lineage harder to reason about. Another mistake is calling cache without any action and claiming data is already stored. Cache is lazy too.

Professional boundary

Cache intentionally, materialize knowingly, inspect actual reuse/storage, and unpersist when the reuse window ends. Never use cache as a reflexive line in every pipeline.

Fresh retry

A cached DataFrame is 60 GB but executors have limited memory. Explain how spill/eviction can reduce benefit and why MEMORY_ONLY thinking from old tutorials may not fit the default DataFrame cache behavior.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-14 | Parquet I/O and partition pruning

What strong evidence contains

The read returns only rows for 2026-09-03 and the requested columns. The plan should show Parquet/file-scan information including partition filters when applicable. Real Spark/Parquet runtime evidence must be collected on the learner PC; static checks alone do not prove scan behavior.

Reasoning to repair

Physical layout gives the engine opportunities to avoid work, but Spark still decides a plan. Projection reduces column I/O; partition filters can avoid whole directory partitions; Parquet statistics can help skip lower-level chunks. These are distinct mechanisms and should not be collapsed into one slogan.

Most important failure to catch

Partitioning by high-cardinality order_id would create tiny directories/files and metadata overhead. Another error is thinking where(year(order_date)==2026) always prunes a directory partition exactly like a direct partition-column predicate; verify the plan. Do not infer pruning from fast execution on 18 rows.

Professional boundary

Choose partition layout from stable access patterns and volume, keep file sizes reasonable, and verify pruning/projection in the actual plan. Logical output checks stay identical regardless of physical layout.

Fresh retry

The dominant query shifts from daily date slices to customer-history lookups. Explain why the old physical layout may no longer be ideal and why you should measure before repartitioning historical data.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-15 | Built-in functions vs Python UDFs

What strong evidence contains

Both versions can produce the same uppercase values. The built-in plan stays within Spark SQL expression machinery; the Python UDF plan exposes a Python UDF evaluation boundary. This is a plan-level teaching observation, not a promise of a fixed speed ratio.

Reasoning to repair

Built-ins give Spark richer semantics for optimization, code generation and execution. Python UDFs require data to cross into Python workers and reduce what Catalyst can see. That overhead matters especially at scale, but correctness still comes first: use the expression that accurately represents the rule.

Most important failure to catch

Do not contort a clear transformation into unreadable built-ins merely to avoid every UDF. Also do not define a UDF for simple tasks Spark already supports, such as uppercasing, date extraction or conditional columns. And never forget a return type/NULL behavior contract.

Professional boundary

Prefer optimizer-visible built-ins when they express the rule clearly. Use UDFs for genuine gaps, with explicit types, null/error behavior, tests and plan/runtime evidence.

Fresh retry

A Python UDF sometimes throws on null input. Design a test and repair contract without silently replacing null with a business value that was never specified.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-16 | Driver safety: collect, toPandas and bounded inspection

What strong evidence contains

The aggregate returns 25,230 and the bounded inspection returns at most five Row objects. No full 18-row collect is necessary even in the toy lab because we are building the habit that scales. A small result after a distributed aggregation is different from dragging raw data to Python for computation.

Reasoning to repair

Driver memory is separate from total cluster capacity. Distributed operations keep partitions on executors; collection centralizes them. The safest pattern is reduce/filter/aggregate/write in Spark, then bring back only a known-small diagnostic or scalar result.

Most important failure to catch

`df.limit(5).toPandas()` is bounded; `df.toPandas().head()` is not, because the full DataFrame reaches the driver before pandas takes the head. Likewise, writing `len(df.collect())` to count rows is far worse than `df.count()`. API order matters.

Professional boundary

Keep large data distributed. Materialize on the driver only after a proven bound or when the API inherently returns a small scalar. Treat driver-memory assumptions as explicit contracts.

Fresh retry

You need to email a 50-row quality report. Explain a safe Spark path that proves the result is ≤ 50 rows before converting to pandas, and what you would do if the bound fails.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-17 | Testing and debugging Spark transformations

What strong evidence contains

The correct test prints PASS. The schema and four rows match. When you inject an incorrect expected value, the testing utility identifies mismatching rows rather than relying on visual inspection. This is stronger evidence than a screenshot of `show()`.

Reasoning to repair

Tiny tests catch transformation logic before cluster cost and data volume hide the problem. They also make refactoring safer: a change to join/partition strategy should preserve logical invariants. Runtime/performance tests are separate; a unit test cannot prove production scaling.

Most important failure to catch

Do not assert incidental row order unless the transformation contract includes ordering. Do not use the same buggy production function to generate 'expected' data. And do not replace tests with one final row count: a fan-out can be followed by a filter that returns the expected count but wrong identities/totals.

Professional boundary

Test transformations on tiny deterministic data, separate logical tests from performance tests, and save failed assertion evidence before repair. A plan refactor should not silently alter business invariants.

Fresh retry

A DataFrame contains floating calculations. Explain when approximate tolerance is appropriate and why loosening tolerance cannot be used to hide a true business discrepancy.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.

DE12-18 | Integrated PySpark pipeline and distributed-processing checkpoint

What strong evidence contains

Source=18; Completed=16; Completed gross=25,230. Region East=5,950, North=6,380, South=6,000, West=6,900. Category Accessories=16,830, Office=8,400. Unique Completed order IDs=16. Your Spark plan/runtime evidence must be added on the learner PC; the build package supplies only static/simulator/reference validation.

Reasoning to repair

The checkpoint combines all four M12 layers: business semantics, distributed API correctness, physical-plan reasoning and operational safety. None substitutes for the others. A perfect explain plan with wrong totals fails; correct totals produced by collecting everything to pandas also fail the distributed-engineering objective.

Most important failure to catch

Common failure modes are implicit schema, accidental Pending/Cancelled inclusion, join fan-out, blind repartition(1), unbounded collect/toPandas, caching one-use data, claiming AQE/skew improvements without runtime evidence, and reporting the 25,230 control from memory instead of recomputing it from the supplied source.

Professional boundary

A Spark pipeline is mastered when you can rebuild it on changed data, prove output with independent controls, interpret the physical plan/runtime evidence, and explain resource/driver/partition safety. Save the full evidence packet; M13 then adds real Delta/lakehouse transaction semantics.

Fresh retry

Use `practice/assessment/practical/parcel_orders.csv` as a changed domain. Build a Spark summary by region for Completed parcels, prove total gross=6000 and completed rows=7, then perform a fresh retest on `clinic_jobs.csv` where Completed amount=3500 across 6 jobs. Do not reuse hard-coded region names or amounts from the order fixture.

Do not call Mastered yet if...

You can repeat this answer but cannot solve the changed task with review closed, or the lesson requires live Spark evidence you have not executed. Static/simulator evidence proves reasoning/code structure, not a Spark engine run.